

• We want performance + control + flexibility

Control implies that the OS ensures that no application is allowed to access any memory but its own.

flexibility implies that programs should be able to use their address spaces in whatever way they would like.

THE CRUX:

HOW TO EFFICIENTLY AND FLEXIBLY VIRTUALIZE MEMORY

How can we build an efficient virtualization of memory? How do we provide the flexibility needed by applications? How do we maintain control over which memory locations an application can access, and thus ensure that application memory accesses are properly restricted? How do we do all of this efficiently?

• An addition to our general approach of limited direct execution is hardware based address translation

• The hardware transforms each memory access (an instruction fetch, load or store) from virtual address to a physical address where it will be stored.

Assumptions

1. User's address space must be placed contiguously in physical memory.
2. The size of address space $<$ size of physical memory.
3. Each address space is the same size.

An Example

↳ On Right is the address space of process A that loads a value from memory, increment it by 3 & then stores the value back in memory.

In C:

```
void func() {  
    int x = 3000; // thanks, Perry.  
    x = x + 3;    // line of code we are interested in  
    ...  
}
```

It is translated to assembly as:

```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax  
132: addl $0x03, %eax        ;add 3 to eax register  
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

↳ Value of x is in ebx

128: Move value of x to general purpose register eax

132: Add 3 to eax

135: Move updated value in eax back to ebx.

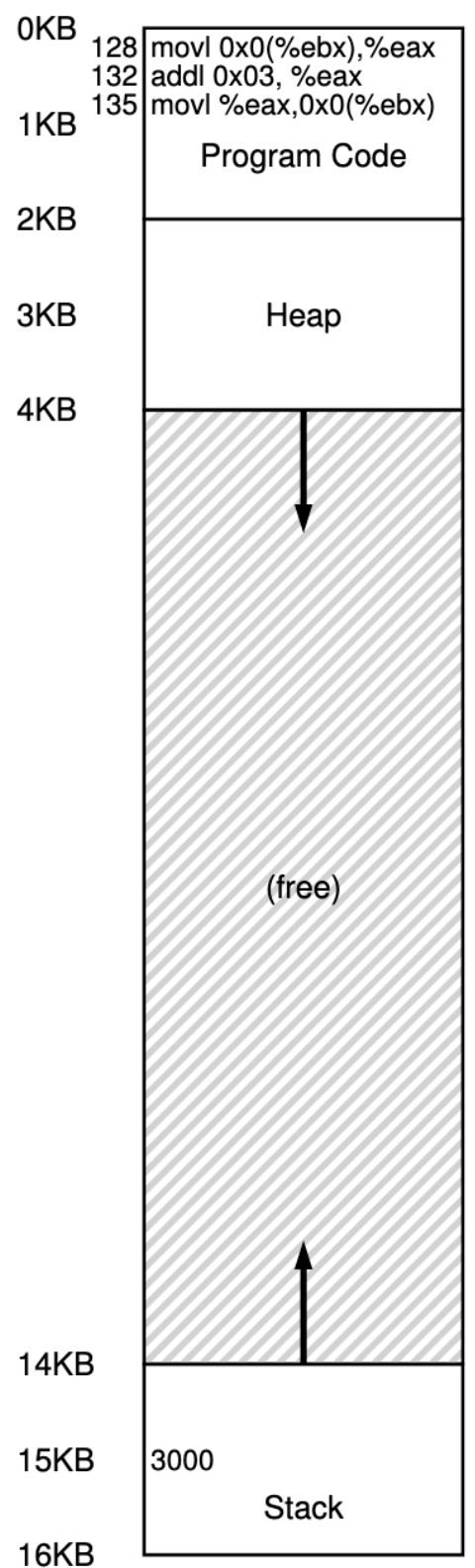


Figure 15.1: A Process And Its Address Space

When these instructions run, from the perspective of the process, the following memory access takes place:

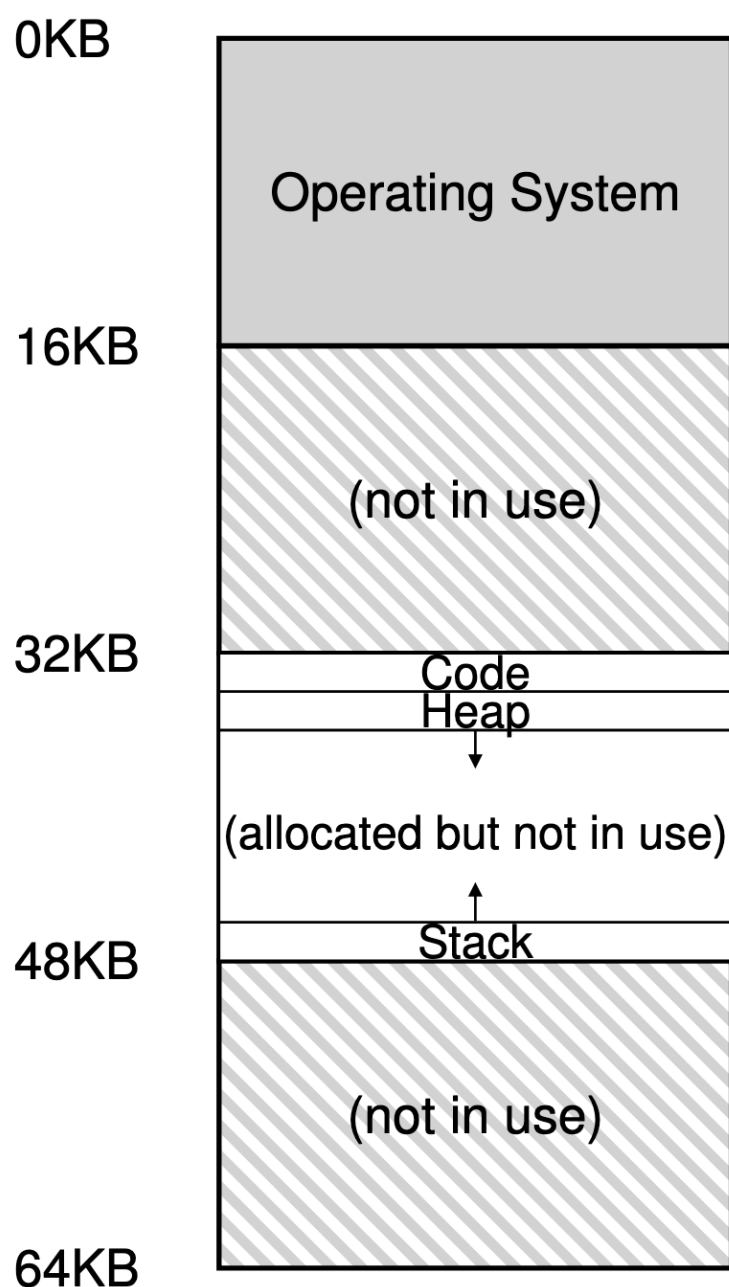
- Fetch instruction at address 128
- Execute this instruction (load from address 15 KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)

From program's perspective, it's address space start at address 0 and grows to maximum of 16KB; all memory references it generates should be within these bounds.

However, to virtualize memory, the OS wants to place the process somewhere else in physical memory, not necessarily at address 0.

So, how can we reallocate this process in memory in a way that is transparent to the process?

An example of how this program is placed in memory is below:



before this was at 0-16kb but now it is here

Dynamic (hardware-based) Relocation

↳ In base & bounds / Dynamic relocation, we will need two registers i.e. base & bounds (limit) registers within each CPU.

↳ In this, a program is loaded & compiled as if it is loaded at address 0.

↳ However, when the program starts running, the OS decides where in physical memory it should be loaded & sets the base register value to that.

In above figure, base register value is 32kB

When the process is running, any memory reference generated by the process is translated as:

$$\text{physical address} = \text{virtual address} + \text{base}$$

For example,

```
128: movl 0x0(%ebx), %eax
```

Here, PC = 128

when hardware needs to fetch this instruction, it first adds the base register value of 32kB (32768) to get a physical address of 32896.

This is referred to as address translation i.e. the hardware takes a virtual address & transforms it into a physical address.

This can be seen as relocating the address & as this happens at runtime it is called dynamic relocation.

The processor will first check that the memory reference is within bounds.

In first figure, limit register will be set to 16kB. If a process generates a virtual address $\geq$ the bound or is negative then CPU will raise an exception.

The part of CPU that helps with address translation is memory management unit (MMU).

NOTE:

Limit register can either hold size of address space & thus hardware checks the virtual address against it before adding the base.

or it holds the physical address of the end of the address space & hardware first adds base & then checks if it is within bounds.

Example,

Assume a process with an address space of size 4kB has been loaded at physical address 16 kB

Result of no. of address translation

Virtual Address		Physical Address
0	→	16 KB
1 KB	→	17 KB
3000	→	19384
4400	→	<i>Fault (out of bounds)</i>

ASIDE: DATA STRUCTURE — THE FREE LIST

The OS must track which parts of free memory are not in use, so as to be able to allocate memory to processes. Many different data structures can of course be used for such a task; the simplest (which we will assume here) is a **free list**, which simply is a list of the ranges of the physical memory which are not currently in use.

Hardware Support

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support address translation and bounds checks</i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i>OS must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i>OS must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or out-of-bounds memory</i>

Figure 15.3: Dynamic Relocation: Hardware Requirements

Operating system issues:

OS Requirements	Notes
Memory management	1. Need to allocate memory for new processes; 2. Reclaim memory from terminated processes; Generally manage memory via free list
Base/bounds management	3. Must ^{save} base/bounds properly upon context switch
Exception handling	Code to run when exceptions arise; likely action is to terminate offending process

Figure 15.4: Dynamic Relocation: Operating System Responsibilities

OS can move a process' address space from one location to another.

To do so OS must,

1. Deschedule the process
2. Copy address space from the current location to the new location
4. Update the saved base registers in PCB

Issues with this

In ②, the process is using physical memory from 32KB to 48KB.

However, because the process' stack & heap are not too big, all of the space b/w them is wasted.

This type of waste is called internal fragmentation.

OS @ boot (kernel mode)	Hardware	(No Program Yet)
initialize trap table	remember addresses of... system call handler timer handler illegal mem-access handler illegal instruction handler	
start interrupt timer	start timer; interrupt after X ms	
initialize process table		
initialize free list		
OS @ run (kernel mode)	Hardware	Program (user mode)
To start process A: allocate entry in process table alloc memory for process set base/bound registers return-from-trap (into A)	restore registers of A move to user mode jump to A's (initial) PC	Process A runs Fetch instruction
	translate virtual address perform fetch	Execute instruction
	if explicit load/store: ensure address is legal translate virtual address perform load/store	(A runs...)
	Timer interrupt move to kernel mode jump to handler	
Handle timer decide: stop A, run B call <code>switch()</code> routine save regs(A) to <code>proc-struct(A)</code> (including base/bounds) restore regs(B) from <code>proc-struct(B)</code> (including base/bounds) return-from-trap (into B)	restore registers of B move to user mode jump to B's PC	Process B runs Execute bad load (illegal memory address)
	Load is out-of-bounds; move to kernel mode jump to trap handler	
Handle the trap decide to kill process B deallocate B's memory free B's entry in process table		